

Ejercicio para el laboratorio del 22S

Entrada/salida con archivos

Para operar con archivos en C++ se utilizan las clases `ifstream` (para lectura) y `ofstream` (para escritura) de la cabecera `#include <fstream>`, como se explicó en el tema 1.

Se pide construir un programa que muestre los préstamos pendientes de una biblioteca. Se dispone de dos archivos, uno (`catalogo.txt`) guarda la información de los ejemplares de los que se dispone la biblioteca. El archivo comienza con un entero indicando el número total de ejemplares, que se describen cada uno en una línea con un código único, un tipo (libro, audiovisual o juego) y un nombre, separados por espacios como se muestra a continuación. Los ejemplares aparecen en el archivo ordenados de menor a mayor código.

```
10
1202 A Pulp Fiction
1203 L Yo, robot
1204 J Resident Evil Requiem
...
```

El otro archivo (`prestamos.txt`) contiene (en ningún orden particular) la relación de préstamos pendientes de devolución (código del ejemplar prestado, fecha de préstamo en formato AA/MM/DD e identificador numérico del usuario). El fichero también comienza con el número de préstamos:

```
7
1204 23/09/26 223
1212 10/09/26 2312
1203 25/08/26 1231
...
```

El programa deberá comenzar cargando la información de cada archivo en las clases `Catalogo` y `ListaPrestamos`, ambas compuestas por un array dinámico de elementos de tipo `Ejemplar` y `Prestamo`, respectivamente, y su tamaño. El catálogo quedará ordenado por orden de códigos (como en el archivo). Una vez leídas las listas, se ordenará la lista de préstamos por fecha de devolución. El programa terminará mostrando la información sobre los préstamos (fecha de devolución, título, días hasta la entrega y penalización por retraso, si la hubiera) como se indica:

```
13/08/26 (en -40 días) Atraco en alta fidelidad (80 días de penalización)
16/09/26 (en -6 días) La guerra de los mundos (12 días de penalización)
23/09/26 (en 1 días) Yo, robot
06/10/26 (en 14 días) Resident Evil Requiem
...
```

Recuerda borrar la memoria dinámica creada.

Tildes en la consola de Windows

En la consola de Windows, para que las tildes, eñes y otros caracteres no ASCII se muestren correctamente es preciso incluir la cabecera `#include <windows.h>` (antes del `using namespace std`, para evitar conflictos de nombres) y llamar en el `main` a la función `SetConsoleOutputCP(CP_UTF8)`.

El código se organiza en cuatro clases, `Catalogo`, `ListaPedidos`, `Ejemplar` y `Prestamo`, con sendos archivos de cabecera (`.h`) e implementación (`.cpp`). Otro archivo `main.cpp` contendrá la función `main`. Añade también al proyecto la clase `Date` que se proporciona en el campus. En la página 4 se muestra un esquema de las clases a implementar con sus posibles atributos y métodos. Un posible guion para implementar la funcionalidad descrita es:

- Crea una `class Ejemplar` con atributos para el código, el tipo y el título del material. Será necesario declarar, antes o dentro de la clase, un enumerado `Tipo` para el tipo con sus tres posibles valores. Ahora, para leer ejemplares desde el archivo `prestamos.txt`, declara en el archivo de cabecera (`.h`) y dentro de la clase un operador

```
friend std::istream& operator>>(std::istream&, Ejemplar&);
```

y defínelo en el archivo de implementación (`.cpp`). Utiliza el operador `>>` sobre tipos básicos o la función `getline` para leer sus atributos. En caso de error se lanzará una excepción.

Funciones amigas

La palabra clave `friend` indica que lo que se declara a continuación no es un método de la clase (es decir, no recibe un objeto implícito) sino una función externa, que sin embargo tendrá acceso a sus atributos y métodos privados. Esto es necesario en funciones como `operator<<` que no pueden ser métodos por definición.

Una forma sencilla de probar la lectura de ejemplares es ejecutar como programa principal

```
#include "Ejemplar.h" // además de <iostream>, etc.
```

```
int main(int argc, char* argv[])
{
    Ejemplar ejemplar;
    std::cin >> ejemplar;
    return 0;
}
```

poner un punto de interrupción en el `return`, escribir una entrada por consola y comprobar que los atributos de `ejemplar` están asignados correctamente.

Errores de lectura en los flujos

La lectura desde un flujo de entrada puede fallar, por ejemplo, si se intenta leer un número pero el carácter que sigue no es numérico. Esta circunstancia se puede detectar con el método `fail` del flujo o usando el propio objeto como un booleano. Si hay un fallo, todas las lecturas siguientes fallarán, a menos que se elimine el marcador de error con el método `clear`.

- Crea una clase `Catalogo` con un array dinámico de `Ejemplares`. Su constructor `Catalogo(std::istream&)` recibirá el flujo del archivo `catalogo.txt`, leerá el número de ejemplares, reservará un array dinámico de ese tamaño y leerá sobre él la información de los ejemplares (usando el `operator>>` sobre `Ejemplar` ya implementado). Esta función no abre el archivo, sino que lo recibe ya abierto.

Ya puedes probar a leer el catálogo entero. El `main` puede abrir el archivo `catalogo.txt` con un objeto `ifstream`, comprobar que se ha abierto correctamente y a continuación construir un objeto `Catalogo` pasándole dicho flujo como argumento. Con el depurador (o un método auxiliar que imprima el catálogo) puedes comprobar que todo se ha cargado correctamente.

- Añade el método `Ejemplar* buscarEjemplar(int) const` al catálogo. El método devolverá un puntero al ejemplar con dicho código o `nullptr` si no existiera tal. Debe implementarse como búsqueda binaria.

Búsqueda binaria

No es necesario implementar la búsqueda binaria porque la biblioteca estándar de C++ la incluye en su cabecera `algorithm`. El método `std::lower_bound` recibe un puntero al inicio del array, un puntero a la posición siguiente a la última, el elemento que buscar y tal vez un comparador (una función). En este caso, como queremos comparar ejemplares con un número de ejemplar, conviene definir un comparador:

```
bool comparaCodigo(const Ejemplar& ejemplar, int codigo) {
    return ejemplar.getCodigo() < codigo;
}
```

y llamar a la función `lower_bound` con `comparaCodigo` como cuarto argumento.

- Crea una clase `Prestamo` con un atributo para el código de usuario, otro atributo de tipo `Date` para la fecha de préstamo y un tercer atributo de tipo `Ejemplar*` con el puntero al ejemplar prestado (que será `nullptr` en caso de no encontrarse el código en el catálogo). La clase tendrá métodos `getter` para consultar el valor de sus atributos privados y un método `void leerPrestamo(const Catalogo&, std::istream&)` que lea un préstamo del flujo recibido como argumento con el formato del archivo `prestamos.txt` y guarde la información en sus atributos. Se requiere también un operador

```
friend std::ostream& operator<<(std::ostream&, const Prestamo&);
```

que imprima la información del préstamo en el flujo de salida con el formato que se observa en la primera página.

- Añade un método `Date getDevolucion() const` al tipo `Prestamo` que calcule la fecha de devolución del material. Como en toda biblioteca, los ejemplares se tienen que devolver algunos días después de haber sido prestados: 30 en el caso de los libros, 7 en el de los materiales audiovisuales y 14 en el de los juegos.
- Crea la clase `ListaPrestamos` como un array dinámico de `Prestamos` y con un constructor/lector `ListaPrestamos(std::istream&, const Catalogo&)`. La lista de préstamos se ha de ordenar por fecha de devolución (de anterior a posterior). Define para ello un método privado `void ordenar()` en `ListaPrestamos` que reordene el array dinámico de préstamos y un operador `bool operator<(const Prestamo&) const` que establezca el orden entre préstamos.

Ordenación en la biblioteca estándar

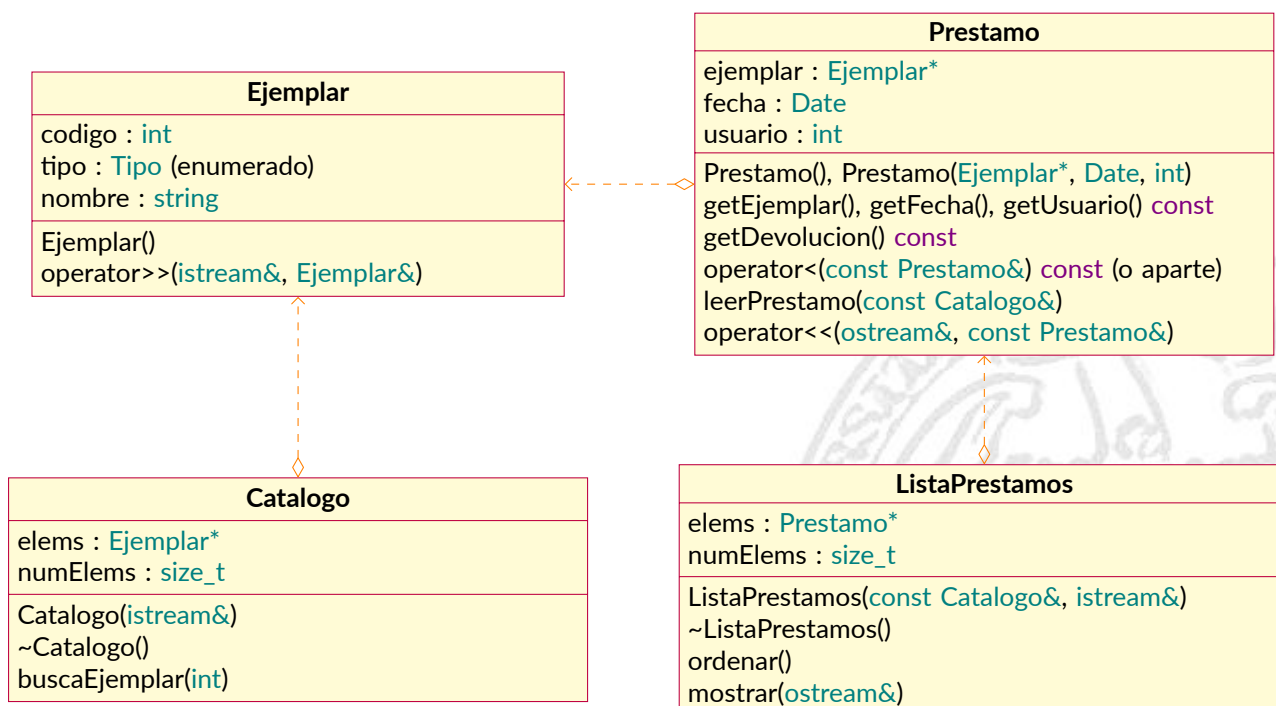
La cabecera `algorithm` de la biblioteca estándar incluye una función `sort` que ordena secuencias de elementos. Recibe tres parámetros: un puntero al comienzo del array, un puntero al primer elemento fuera de él y el nombre de la función que implemente el orden entre elementos. El tercer argumento es opcional y en su defecto se utilizará el operador `<` que esté definido sobre los elementos (si no hay ninguno dará un error de compilación). Puedes definir el operador `<` sobre el tipo `Prestamo` con

```
bool operator<(const Prestamo& otro) const {
    // Definición del orden
}
```

Alternativamente, `bool operator<(const Prestamo& izdo, const Prestamo& dcho)` definido fuera de la clase puede actuar también como comparador (el argumento izquierdo será el `this` implícito).

- Programa el método `void mostrar(std::ostream&)` en `ListaPrestamos` para mostrar la relación de préstamos en el formato ejemplificado arriba. Utiliza para ello el `operator<<` de la clase `Prestamo`.
- Asegúrate de que toda la memoria reservada se libera antes de finalizar el programa (es decir, todo `new` tiene su `delete`). Utiliza destructores en aquellas clases que lo necesiten.

En el CV puedes encontrar un ejemplo de ficheros de entrada y un fichero (`salida.txt`) con la salida que tu programa debería generar para dichos ficheros de entrada. También hay incluida una clase `Date` que representa una fecha del calendario y que puedes utilizar en tu programa principal incluyéndola con `#include "Date.hpp"` y añadiendo `Date.cpp` a la lista a archivos a compilar en el proyecto. Copia todos los archivos, incluidos los `.txt`, al directorio del proyecto de Visual Studio. Un diagrama de las clases que hay que implementar, con más detalles de los posibles atributos y métodos auxiliares, se muestra a continuación.



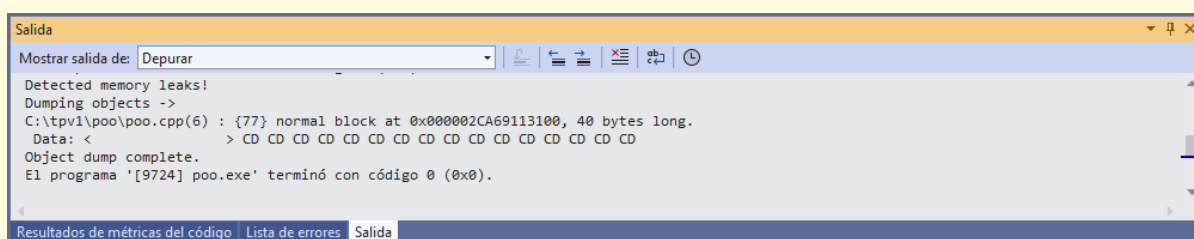
Entrega: se debe realizar una entrega por grupo en la que solo debéis subir vuestros ficheros de código fuente (.h y .cpp, incluyendo en el main.cpp vuestros nombres y el identificador del grupo). No lo comprimáis en zip, subid los ficheros directamente. La entrega se realiza en el CV, en la pestaña *Laboratorio y prácticas*.

Ejercicios adicionales: (no hay que entregarlos, aunque lo que se entregue no debería generar fugas ni errores de memoria)

1. Comprueba que el programa no deje fugas de memoria.

Comprobación de fugas de memoria en Visual Studio

El archivo checkML.h disponible en el campus añade instrumentación a las funciones **new** y **delete** para detectar fugas de memoria al finalizar el programa. Has de incluir la cabecera al principio de cada archivo que reserve memoria dinámica (asegúrate también de que en la configuración del proyecto esté seleccionado el estándar C++20). Las fugas de memoria aparecerán en el panel de salida de Visual Studio.



Si usas g++ o clang++ en Linux o macOS esto no funcionará, pero con la opción `-fsanitize=address` el compilador mostrará también las fugas de memoria al finalizar la ejecución.

2. Intenta guardar una copia de la lista de préstamos leída en el main antes de ordenar.

```
ListaPrestamos(catalogo, archivo); // o equivalente  
ListaPrestamos original = prestamos;
```

¿Qué pasa al finalizar el programa? Redefine los métodos necesarios para evitar este problema.

3. Cambia la representación del catálogo de manera que se haga uso de un array dinámico de punteros a ejemplar. Es recomendable que antes de hacer este cambio pruebes todo lo demás y dejes una versión guardada. Vuelve a comprobar que no haya fugas de memoria.
4. Piensa cómo se podría implementar el préstamo o devolución de ejemplares del catálogo.